

our project: label-efficient disaster detection

The Challenge: Disaster damage assessment relies on manually labeled satellite/drone imagery, which is slow, expensive, and requires domain experts.

Why It Matters:

- Labeling thousands of images takes days to week:

The process is time-consuming and mentally exhausting

••

Not scalable during emergency situations

Delays critical relief planning and resource allocation

Our Focus:

Develop methods that learn effectively from minimal labeled data, enabling faster, scalable, and reliable disaster response.

V Satellite images available in hours

X Expert damage labels take days

Emergency response needs answers NOW

Xod dataset

22068 images, 850000 buildings, 19 disaster types, 0.5m resolution

Color coding suggests that green means least/minor damage and red means completely destroyed. It is like a scale where green means damage < 10% and red means damage > 95%.

What is Siamese Unet?

Siamese U-Net is a dual-branch encoder-decoder network designed for bi-temporal image analysis (e.g., before & after disaster images).

Architecture

- Two identical encoder branches
- Shared weights (Siamese structure)
- Extract features from before and after images
- Features are combined and passed through a decoder
- Produces a pixel-level change map

Why It's Used in Disaster Detection

- Learns structural differences between image pairs
- Effective for building damage localization
- Works well for satellite image change detection

SpaU-Net is a U-Net variant with spatial attention, designed for detecting subtle changes in pre-and post-disaster images.

Key Features

- U-Net backbone for encoder-decoder segmentation
- Spatial attention modules focus on regions likely to have damage
- Can handle small or subtle structural changes that might be missed by standard U-Nets

U-Nets

- Works well with limited labeled data

How It Works

- Input before and after images
- Encoder extracts features from both images
- Spatial attention highlights areas of interest (e.g., damaged buildings)
- Decoder combines features and attention maps to produce pixel-level damage predictions

Why It's Useful

- Improves detection of subtle damages
- Enhances performance when only a few labeled images are available

Complements Siamese U-Net by adding attention-driven focus on critical regions

So we try to implement, uh, two of the good solutions that we found in our domain, which is label efficient. Um, the first one was the I mean, the data set to be used here, which was xbd, it was actually, uh, It was actually given by Xview2 in a challenge as a data set to use for the participants to come up with a solution. So we first tried to reproduce the, uh, the solution that actually won the competition, because they do not have a paper on this, but they do have a GitHub repository. So that's what we tried to do. And the second one was disaster adaptive net, which is a recent paper which uh did does not uh beat the first place solution in terms of accuracy, but it is the more efficient version. Um, so our attempt, uh, we only changed a little bit of stuff like, uh Trained and tuned all models only once instead of thrice as opposed to actual approach. (trained 12 models instead of 42 due to time and resources limitation)

- Modified scripts slightly to work better in our setup (Linux with Cuda using Nvidia L40S GPU, original approach used 2 GPUs)
- Implemented Apex approach using torch, and replaced some deprecated libraries/methods (like numpy square)
- No change to models, dataset, epochs, hyperparameters etc.
- Achieved total score on test: 0.788 (as opposed to actual solution's 0.8)

Uh, like we got 0.788 score, which that was proposed, was appointed and this was one test, uh, uh test split.

And for the second one, uh, for disaster adaptive net.

- Tested reproducibility of DisasterAdaptiveNet paper (Hafner et al., 2025)
- Downloaded pre-trained model weights from authors' provided links
- Processed data and generated masks from JSON labels

- Tested on 2 different types of models: disasteradaptive-net-event, strong-baseline-event
 - Produced localization and damage classification outputs
 - Score of 0.770 proposed by the papers for Xbd dataset used in Xview2 which had overlapping splits.
 - Score of 0.541 proposed by the papers for Xbd dataset with event based split.
- Uh, we were mistaken to think that we did not, uh, get, uh, proposed results. We were not able to reproduce them. But the thing was, uh, the we were actually testing it on the Uh, no, the weights that were given by the models were actually, uh, they followed the different type of split like in the original actually data set, it has a split which, like, uh, randomly strikes our data. So what happens is if there's an event, let's say tornado in 2011 and Joplin. So that event will contain some images in test and some images in train. So that's why, uh, to get more accuracy with that, as they said, they performed well and gave point seven score on that dataset. But they also introduced another split, uh, in our paper, which was result of the same authors, uh, they introduced a new split set which was said that event based, uh, splitting. So if one event is present in train, it will not be in test. So the model will be truly looking at unseen, like just like how it will work in real life. And uh, and on that split, they, they were only able to get 0.541 score. So that's what we did produce. We thought we were trying to reproduce the first one, but we were basically doing the second. And this for our, uh, results, which is quite similar to what they got.

Experiment 1: DisasterAdaptiveNet event based

"score": 0.5330747043197341,
 "damage_f1": 0.42351801014369883,
 "localization_f1": 0.7887069907304834,
 "damage_f1_no_damage": 0.7763602596133505,
 "damage_f1_minor_damage": 0.25306230599661816,
 "damage_f1_major_damage": 0.3370439474177524,
 "damage_f1_destroyed": 0.8076915807901398

Experiment 2: Strong Baseline event based

"score": 0.47787474653018114,
 "damage_f1": 0.340557689046936,
 "localization_f1": 0.7982812139910864,
 "damage_f1_no_damage": 0.801704252765164,
 "damage_f1_minor_damage": 0.17102141262529746,
 "damage_f1_major_damage": 0.2921386202708222,
 "damage_f1_destroyed": 0.8144032537096821

They got about 0.41 and we got 0.533 for the first one.

- DisasterAdaptiveNet outperforms baseline (+11% overall)

- FiLM conditioning improves damage classification
- Both models struggle with minor/major damage classes

All major majority of the models that are being used to, uh, solve these problems. They use a pre-trained model for first localization, and then they use their weights for classification for like, damage prediction.

Um, our current progress, we actually produced, uh, two of the papers

Reproducibility Study 1 - xView2 First-Place Solution

- Implemented two-stage pipeline: building localization → damage classification
- Trained 12 backbone models (ResNet-34, ResNet-50, DPN-92, SENet-154)

(original paper used 42 models)

- Adapted implementation for CUDA + NVIDIA L40S GPU

Result:

- Score achieved: 0.788
- Original reported: 0.812

Reproducibility Study 2 - DisasterAdaptiveNet

- Used author pretrained weights
- Generated masks from dataset JSON annotations
- Tested models:

: sangeradatetevent

Results

- Best score: 0.533
- Localization F1: ~0.79

Lower accuracy for minor / major damage classes We actually looked through some foundational models, uh, that we can use, uh, for our localization task. And, uh, we were trying to find a model that was pre-trained on a data set whose resolution matched with our data set resolution, which was about 0.5 meter and whose weights were publicly available as well. Because, to be honest, we do not have time or resource to train a foundational model. So we came up with this and we tried to like analyze which would be compatible for our use case. And this is what we got. Um, you can see here the crossearch is something that we don't have. Crossearch utilizes Dino V2 And uh, seeing this table, we can see that SATLAS, skysense and crossEarth are some models that potentially can give us good results. when we were researching about these models, we were parallely also, uh, experimenting with model segformer and dinov2, which we knew beforehand. So when we try to fine tune segformer for the research, And for the localization F1, that's the result. We are really concerned with the 0.628 localization F1, which is not a good. And similarly for Dino V2, uh, we did a dry run and then we fine tuned it and there was not much improvement in the resolution of the fine tuning. DinoV2 Run:

- Localization task dry run:
- Overall dice score: 48.97%
- Images evaluated: 1866
- Fine-tuned model Validation metrics:
- Validation dice: 53.61%
- Validation IoU: 36.62%

And this is also not good. It's just 53.61% dice score. But for satlas we were able to get some promising results. We got 0.8191 validation. We got 0.8191 validation. we actually first uh downsampled. Because there's like, uh, mismatch between tbd data set image resolution and uh satlas input image resolution. So we first downsampled and tried and accuracy was not good of course. So then we did tiling. And we actually saw that accuracy improved by a factor of 0.1 right around with this. Like it was 0.7 at first. Now we have 0.8 um, and uh, another issue was that the building pixel ratio is just 6% in xbd data set. Um, so we use focal loss to like handle class imbalance, uh, automatically. And then um, uh, segmentation had an which was given in the repository of this satlas model, but it was randomly initialized. So we first trained only that for five epochs and froze every other layer. And then we unfroze everything. But we made sure that the learning rate of the upsampled and the segmentation was still more than other layers. And this is what we ended up with. SATLAS Run:

Fine-tuned model Validation metrics for localization task:

- Validation dice: 0.8191
- Validation IoU: 0.7466

Model Training Specs:

- Xbd dataset resolution - 1024X1024, Model accepts - 512X512. Solution: tiling.
- Xbd dataset building pixel ratio: approx. 6%. Focal loss used to handle class imbalance
- Segmentation head is randomly initialized. Hence, trained head ONLY for 5 epochs and then unfroze everything and trained for another 10 epochs.
- LR for head and upsample layer: 10^{-4} , LR for other layers: 10^{-5} in disaster adaptive net, Uh, what this paper did is that, uh, they also I had a previous paper regarding this problem where they introduced an event based split in xbd, they split images randomly. Like test and train will contain images from the same event which is possible in xbd dataset. But they introduce an event based split. Where is, uh, the training data set has one event, then testing will have an completely different event. So that's how it would work in real life. So they did that and they tried that approach. Disaster adaptive net on both of the data sets xbd and the event based. We got 0.77 on xbd and 0.54 on event split. And when we try to reproduce that, we kind of got close to it we got 0.53. Um, yeah. So this was our second reproducibility event. Um, what we did notice is that both of the solutions usually struggle with minor and major damage classes. Possibility is because there might be a class imbalance that might be, uh, low. I

mean, less number of data points of minor major damage classes. Um, then we moved on to phase one of our project, which was, um, investigating, comparing and then utilizing, uh, foundational models for localization. Um, so this this was some of the, uh, foundation model that we shortlisted for our task. Uh, we made sure that we only chose those models whose public weights are available so that we don't have to train them and image resolution also we tried to chose similar to xbd which was 0.5m. And these are some example images that we got when we predicted on the atlas. Uh, it we noticed that it was kind of, uh, um, over localizing, I think. So it was like it was predicting some stuff which wasn't as buildings. Um, as we can see here, the prediction map is more dense as compared to the ground truth, and it's also not as fine. Uh, it does not have that fine details, uh, close boundaries that are closer to each other are merged. So moving forward in the spring break, we tried with, uh, deep labs implementation, and we also ran, uh, same implementation to see how that works. And we ran this deep labs architecture for 30 epochs. And, uh, this was the best model that we have as of now. So for this model, uh, after 30 epochs, we got, uh, validation dice of point 87 and validation iou of point 78. And, uh, this model basically uses ResNet 52. So we tried with two different backgrounds and we got best results with, uh, ImageNet, backbone. And so uh, to check with like how sam runs for our task. We, uh, ran same with the ground truth boxes, like the prompts that were given in the Xview2 data set. And we also used it with, uh, prediction that we are doing with, uh, deep labs implementation. So with ground truth, we are getting, like, very good, uh, validation, IOU and dice scores But, uh, for like, uh, prediction predicted samples, we were not getting as good as we used iou and dice. So now we tried with Sam, sam2, samgeo in this where the scores that we got.

DeepLabsV3+ Run:

- Fine-tuned ImageNet backbone model Validation metrics for localization task:
- Validation dice: 0.8772
- Validation IoU: 0.7812
- Fine-tuned FMoW backbone model validation metrics for localization task:
- Validation dice: 0.8729
- Validation IoU: 0.7745
- Model Training Specs:
- Xbd dataset resolution - 1024X1024, Model accepts - 512X512. Solution: tiling.
- Loss function: BCE with Logits loss
- LR of 1e-4 Okay. And so very recently we reran the dino V2 for 30 epoch. And

after the training was done we got around point 77 validation iou, which was the second best score that we have as of now. So as you can see the it is doing even better than the ground truth in some cases where like in ground truth, the two buildings are like having a single bounding box, whereas in prediction we can see the model is predicting them as two separate buildings. But in some cases, like when it is like a very dense buildings, in some cases it is not doing like that. And so yeah, basically so the uh, problem here is

we are trying to solve is that,uh, natural disasters can cause like billions of dollars in damages annually.So, uh, we need a method that rapidly assesses the damage that happens from, uh, based on the satellite imagery.And so, uh, as of now, the responders have to manually go and map down the damages and which takes, uh, basically, uh, weeks and more than that.So basically speed is an issue. And also granularity like we do not want that a specific area is damage.So to handle this uh, issue, we are using a data set which has more than 850,000 uh, building polygons with uh, ranging around, uh, 19 disaster events.And all of those has pre and post images and like they have for uh, damage severity labels, uh, ranging from no damage to uh, uh, destroyed buildings.And so, uh, these are the existing solutions that we found. The first one was xview2 first place solution and the second one was disasteradaptivenet, And the first one was, uh, in the first, uh, solution, they proposed their pipeline into two stages.The first one was like they were localizing the images, and based on that, they were classifying it.And for that, uh, they used four backbones and simple like ResNet 34, ResNet 50, ResNet 92 and ResNet 154 and with different, uh, seed values and different variations, they ensembled, and they got around like 42 models and out of which they were like, uh, getting their results. So their mean F1 score was around 0.804. And so that was that is the state of the art result for this data set.And the second one which we got was a disaster adaptive net, and for which they published a paper in 2025 in a journal.And so for that they uh, provided a normal split like the normal design results using the split, which were provided by xview2 competition.And also they provided their own splits where the data which is in training data like the disaster which is in training data would not be in the testing data. So, uh, basically that's what happens in the normal cases.So for that they, uh, got uh, f1 score of 0.770.we identified that we should go with a two phase approach where we use uh for as a first phase, we use foundation models to localize the buildings.And in the second phase we use those models for like, uh, damage classification.And uh, for the first phase, we identified around seven geospatial, uh, foundation models, and we fine tuned like,uh, uh, shortlisted models, which were like compatible with, uh, resolutions of xbd data set. And, and the best model that we got out of phase one, we used it for, uh, phase two.And for the phase two, we use, uh, dual branch like, uh, pre plus post disaster image pairs as an input and uh, five class output, basically background plus 4 for uh severity levels. And for the phase one, we got best results from Deep labs V three plus architecture. And that is what we used along with, uh, ResNet 50 backbone for our phase two.And for uh, reproducibility studies, uh, for the first x view2 first place, the uh reported uh score of 0.804 and where we got our score as 0.788, which was quite low. But like, uh, also they used 42 uh, models where we only trained 12 models. So that's what explains the gap between the results. And also for, uh, Disaster adaptive net. We use that event based splitting.So wherever you are, like using two models, one was like uh, uh, baseline that that they used.And the second one was uh, their

approach. And so the weights were already available. So we already just did a basic few changes to the code. And we ran as it is, we did not change anything. So they reported, uh, event based splitting score of 0.541. And we got uh, as 0.533. And for the baseline score, they reported 0.483. And when we got like 0.477. So yeah, this was like the gaps were explained. This is the basic comparison that we did. I'll be walking you through our, uh, first step of the approach. First one was working with foundational models to get to, like, find which model works the best and localizing. So that's what we are trying to do. And not only find we are trying to pre-train it, uh, on the xbd data so that, uh uh, we can get. So when it moves on to classification step, it already knows how to identify buildings. So we started with first. Well, of course looking at the foundational models, we looked at a lot of models. And we shortlisted some of them um, here we uh, actually were looking for, uh, I mean, whenever we were looking for any foundation models, we asked ourselves three questions, like, is this particular model trained on geospatial data? Uh, is it, uh, ready for segmentation? Uh, is it like, uh, is the resolution that this data was trained that this model was trained on matched our data? So that's what we're looking for, though very few matches. Uh, one perfect match that we got was for satlas pre-trained, uh, where you can see it checks all of our questions. Uh, and also Deep Labs v3, it partially checks your questions. Um, to our surprise, satlas did not perform well. It was not one of the best models that we, uh, got out of our experiments in phase one. Uh, and, uh, Dino V2, on the other hand, which as well, which was not pre-trained on geospatial data, at least not one version of it. Uh, that actually gave us better results than satlas. And at least that was quite unexpected. So we actually ran all of these models, uh, um, like, as our experiment in phase one. Uh, our best was Deep Labs V3 with ImageNet backbone. So it had like two backbones. One was ImageNet and another was functional map of world, which is, uh. Well, which is a geospatial data set. So we actually expected it to work better, but uh, it was quite the opposite, ImageNet model outperformed the flow backbone model. It was not like significant improvement in performance, but it's still quite surprising. And uh it's this performance is quite comparable to the first place solution that we looked at. Um, and, uh, yeah. So resolution that, uh, the data set has is 110241024 and the model accepts 512512 resolution. So we did tiling and sort of downsampling it. Uh, we we did naively downsample it for dinoV2 runs, but, uh, that we ended up getting horrible performance along with some of the mistakes that we made. So that's why we also did dinoV2 reruns, uh, with the same experiments that we did in deep labs. And we this is like the second best performing model that we got dinov2. Uh, it has 0.874 validation dice. Not not like it's. It's not that significantly different. Just 0.003. And all the experiments were less or done in the same, um, configuration other than. Deep labs Uh like they are above the um among the top models that we got then Dino V2 of course was one Sam models. I don't think it's a valid comparison here because well we use ground truth problems which is kind of cheating. And cross earth also gave good performance, but it's

not the best. Uh, it was doing good when there's less building in the images, but it was not doing so good when you know the images for dense for those buildings. So we used dual branch, uh, deep lab v3, because, uh, in that model, we got the best results from doing experiments during the phase one. Uh, so the thing is that what we are reporting here, the mean F1, that that's only for damage classification, uh, usually uh, and then uh, and there are solutions dealing with. Xbd. They actually uh report in xbd test code, which is a mixture of localization and damage classification. But uh, our score kind of, uh, well, it is a little better than the xview2 winner because their damage test score was 0.779. Uh, we did found, uh, we did found an, uh, approach that was outperforming ours. But, uh, but the latest, uh, paper that we are looking, uh, looking at, they did the comparison analysis on this paper and their rerun did lower than us. So that's what we have to look into right now. That is our, our our results. Like actually good enough or, you know, uh, if some other solution before us have already outperformed it. We have like two decoder heads. One is uh, building localization and model is building, uh, damage classification. So it is kind of multitask learning. Uh, so we will be using that to, uh, get the final xbd test score for, uh, us.